

# Spring Boot集成ELK

参考链接：

[https://github.com/qos-ch/logback/tree/v\\_1.5.18/logback-examples](https://github.com/qos-ch/logback/tree/v_1.5.18/logback-examples)

<https://github.com/logfellow/logstash-logback-encoder/tree/logstash-logback-encoder-9.0>

## 1 分场景配置

为了方便我们查看日志，我们把日志分为以下四种：

- 调试日志：最全日志，包含了应用中所有DEBUG级别以上的日志，仅在开发、测试环境中开启收集；
- 错误日志：只包含应用中所有ERROR级别的日志，所有环境只都开启收集；
- 业务日志：在我们应用对应包下打印的日志，可用于查看我们自己在应用中打印的业务日志；
- 记录日志：每个接口的访问记录，可以用来查看接口执行效率，获取接口访问参数。

下面来看一个完整的配置示例，配置文件命名为logback-spring.xml，放在classpath下（resources根目录）。

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE configuration>
3 <configuration>
4     <!-- 引用默认日志配置 -->
5     <include resource="org/springframework/boot/logging/logback/defaults.xml"/>
6     <!-- 使用默认的控制台日志输出实现 -->
7     <include resource="org/springframework/boot/logging/logback/console-
8 appender.xml"/>
9     <!-- 日志文件保存路径 -->
10    <property name="LOG_FILE_PATH"
11 value="\${LOG_FILE:-\${LOG_PATH:-\${LOG_TEMP:-\${java.io.tmpdir:-/tmp}}}/logs}" />
12    <!-- 项目名称 -->
13    <springProperty name="PROJ_NAME" scope="context"
14 source="application.title" defaultValue="01-star" />
15    <!-- 应用名称 -->
16    <springProperty name="APP_NAME" scope="context"
17 source="spring.application.name" defaultValue="spring-
18 boot" />
19    <!-- LogStash访问host -->
20    <springProperty name="LOG_STASH_HOST" scope="context"
21 source="logstash.host" defaultValue="localhost" />
22    <!-- DEBUG日志输出到文件 -->
23    <appender name="FILE_DEBUG"
24 class="ch.qos.logback.core.rolling.RollingFileAppender">
25        <!-- 输出DEBUG以上级别日志 -->
26        <filter class="ch.qos.logback.classic.filter.ThresholdFilter">
27            <level>DEBUG</level>
28        </filter>
29        <encoder>
30            <!-- 设置为默认的文件日志格式 -->
```

```

30         <pattern>${FILE_LOG_PATTERN}</pattern>
31         <charset>UTF-8</charset>
32     </encoder>
33     <rollingPolicy
class="ch.qos.logback.core.rolling.SizeAndTimeBasedRollingPolicy">
34         <!-- 设置文件命名格式 -->
35         <fileNamePattern>${LOG_FILE_PATH}/debug/${APP_NAME}-%d{yyyy-MM-dd}-
%i.log</fileNamePattern>
36         <!-- 设置日志文件大小，超过就重新生成文件，默认10M -->
37         <maxFileSize>${LOG_FILE_MAX_SIZE:-10MB}</maxFileSize>
38         <!-- 日志文件保留天数，默认30天 -->
39         <maxHistory>${LOG_FILE_MAX_HISTORY:-30}</maxHistory>
40     </rollingPolicy>
41 </appender>
42
43 <!-- ERROR日志输出到文件 -->
44 <appender name="FILE_ERROR"
45     class="ch.qos.logback.core.rolling.RollingFileAppender">
46     <!-- 只输出ERROR级别的日志 -->
47     <filter class="ch.qos.logback.classic.filter.LevelFilter">
48         <level>ERROR</level>
49         <onMatch>ACCEPT</onMatch>
50         <onMismatch>DENY</onMismatch>
51     </filter>
52     <encoder>
53         <!-- 设置为默认的文件日志格式 -->
54         <pattern>${FILE_LOG_PATTERN}</pattern>
55         <charset>UTF-8</charset>
56     </encoder>
57     <rollingPolicy
class="ch.qos.logback.core.rolling.SizeAndTimeBasedRollingPolicy">
58         <!-- 设置文件命名格式 -->
59         <fileNamePattern>${LOG_FILE_PATH}/error/${APP_NAME}-%d{yyyy-MM-dd}-
%i.log</fileNamePattern>
60         <!-- 设置日志文件大小，超过就重新生成文件，默认10M -->
61         <maxFileSize>${LOG_FILE_MAX_SIZE:-10MB}</maxFileSize>
62         <!-- 日志文件保留天数，默认30天 -->
63         <maxHistory>${LOG_FILE_MAX_HISTORY:-30}</maxHistory>
64     </rollingPolicy>
65 </appender>
66
67 <!-- DEBUG日志输出到LogStash -->
68 <appender name="LOG_STASH_DEBUG"
69     class="net.logstash.logback.appender.LogstashTcpSocketAppender">
70     <filter class="ch.qos.logback.classic.filter.ThresholdFilter">
71         <level>DEBUG</level>
72     </filter>
73     <destination>${LOG_STASH_HOST}:4560</destination>
74     <encoder charset="UTF-8"
class="net.logstash.logback.encoder.LoggingEventCompositeJsonEncoder">
75         <providers>
76             <timestamp>
77                 <timeZone>Asia/Shanghai</timeZone>
78             </timestamp>
79             <!-- 自定义日志输出格式 -->
80             <pattern>
81                 <pattern>
82                     {

```

```

82         "project": "${PROJ_NAME:-}",
83         "level": "%level",
84         "service": "${APP_NAME:-}",
85         "pid": "${PID:-}",
86         "thread": "%thread",
87         "class": "%logger",
88         "message": "%message",
89         "stack_trace": "%exception{20}"
90     }
91     </pattern>
92 </pattern>
93 </providers>
94 </encoder>
95 <!-- 当有多个LogStash服务时，设置访问策略为轮询 -->
96 <connectionStrategy>
97     <roundRobin>
98         <connectionTTL>5 minutes</connectionTTL>
99     </roundRobin>
100 </connectionStrategy>
101 </appender>
102
103 <!-- ERROR日志输出到LogStash -->
104 <appender name="LOG_STASH_ERROR"
105 class="net.logstash.logback.appender.LogstashTcpSocketAppender">
106     <filter class="ch.qos.logback.classic.filter.LevelFilter">
107         <level>ERROR</level>
108         <onMatch>ACCEPT</onMatch>
109         <onMismatch>DENY</onMismatch>
110     </filter>
111     <destination>${LOG_STASH_HOST}:4561</destination>
112     <encoder charset="UTF-8"
113 class="net.logstash.logback.encoder.LoggingEventCompositeJsonEncoder">
114         <providers>
115             <timestamp>
116                 <timeZone>Asia/Shanghai</timeZone>
117             </timestamp>
118             <!-- 自定义日志输出格式 -->
119             <pattern>
120                 <pattern>
121                     {
122                         "project": "${PROJ_NAME:-}",
123                         "level": "%level",
124                         "service": "${APP_NAME:-}",
125                         "pid": "${PID:-}",
126                         "thread": "%thread",
127                         "class": "%logger",
128                         "message": "%message",
129                         "stack_trace": "%exception{20}"
130                     }
131                 </pattern>
132             </pattern>
133         </providers>
134     </encoder>
135     <!-- 当有多个LogStash服务时，设置访问策略为轮询 -->
136     <connectionStrategy>
137         <roundRobin>
138             <connectionTTL>5 minutes</connectionTTL>
139         </roundRobin>

```

```
138     </connectionStrategy>
139 </appender>
140
141 <!-- 业务日志输出到LogStash -->
142 <appender name="LOG_STASH_BUSINESS"
143 class="net.logstash.logback.appender.LogstashTcpSocketAppender">
144     <destination>${LOG_STASH_HOST}:4562</destination>
145     <encoder charset="UTF-8"
146 class="net.logstash.logback.encoder.LoggingEventCompositeJsonEncoder">
147         <providers>
148             <timestamp>
149                 <timeZone>Asia/Shanghai</timeZone>
150             </timestamp>
151             <!-- 自定义日志输出格式 -->
152             <pattern>
153                 <pattern>
154                     {
155                         "project": "${PROJ_NAME:-}",
156                         "level": "%level",
157                         "service": "${APP_NAME:-}",
158                         "pid": "${PID:-}",
159                         "thread": "%thread",
160                         "class": "%logger",
161                         "message": "%message",
162                         "stack_trace": "%exception{20}"
163                     }
164                 </pattern>
165             </pattern>
166         </providers>
167     </encoder>
168     <!-- 当有多个LogStash服务时，设置访问策略为轮询 -->
169     <connectionStrategy>
170         <roundRobin>
171             <connectionTTL>5 minutes</connectionTTL>
172         </roundRobin>
173     </connectionStrategy>
174 </appender>
175
176 <!-- 接口访问记录日志输出到LogStash -->
177 <appender name="LOG_STASH_RECORD"
178 class="net.logstash.logback.appender.LogstashTcpSocketAppender">
179     <destination>${LOG_STASH_HOST}:4563</destination>
180     <encoder charset="UTF-8"
181 class="net.logstash.logback.encoder.LoggingEventCompositeJsonEncoder">
182         <providers>
183             <timestamp>
184                 <timeZone>Asia/Shanghai</timeZone>
185             </timestamp>
186             <!-- 自定义日志输出格式 -->
187             <pattern>
188                 <pattern>
189                     {
190                         "project": "${PROJ_NAME:-}",
191                         "level": "%level",
192                         "service": "${APP_NAME:-}",
193                         "class": "%logger",
194                         "message": "%message"
195                     }
196                 </pattern>
197             </pattern>
198         </providers>
199     </encoder>
200 </appender>
```

```

192         </pattern>
193     </pattern>
194 </providers>
195 </encoder>
196 <!-- 当有多个LogStash服务时，设置访问策略为轮询 -->
197 <connectionStrategy>
198     <roundRobin>
199         <connectionTTL>5 minutes</connectionTTL>
200     </roundRobin>
201 </connectionStrategy>
202 </appender>
203
204 <!-- root appender DEBUG级别以上日志 -->
205 <root level="DEBUG">
206     <!-- 控制台日志 -->
207     <appender-ref ref="CONSOLE" />
208     <!-- 调试日志 -->
209     <appender-ref ref="LOG_STASH_DEBUG" />
210     <!-- 错误日志 -->
211     <appender-ref ref="LOG_STASH_ERROR" />
212 </root>
213 <!-- 业务日志 -->
214 <logger name="com.zeroone.star.projectlog" level="DEBUG">
215     <appender-ref ref="LOG_STASH_BUSINESS" />
216 </logger>
217 <!-- 记录日志 -->
218 <logger name="com.zeroone.star.projectlog.component.WebLogAspect"
219 level="DEBUG">
220     <appender-ref ref="LOG_STASH_RECORD" />
221 </logger>
222
223 <!-- 控制框架输出日志 -->
224 <logger name="org.slf4j" level="INFO" />
225 <logger name="springfox" level="INFO" />
226 <logger name="io.swagger" level="INFO" />
227 <logger name="org.springframework" level="INFO" />
228 <logger name="org.hibernate.validator" level="INFO" />
</configuration>

```

后面我们分开来对每个配置要点进行解释。

## 2 Logback配置

### 2.1 控制台日志配置

一般我们不需要自定义控制台输出，可以采用默认配置。

具体配置参考console-appender.xml，该文件在spring-boot-\${version}.jar下面。

```

1 <!-- 引用默认日志配置 -->
2 <include resource="org/springframework/boot/logging/logback/defaults.xml" />
3 <!-- 使用默认的控制台日志输出实现 -->
4 <include resource="org/springframework/boot/logging/logback/console-
  appender.xml" />

```

## 2.2 springProperty

该标签可以从Spring Boot的配置文件中获取配置属性，比如说在不同环境下我们的Logstash服务地址是不一样的，我们就可以把该地址定义在application.yml来使用。

例如在application-dev.yml中定义了这些属性：

```
1 logstash:
2   host: localhost
```

在logback-spring.xml中就可以直接这样使用：

```
1 <!-- 应用名称 -->
2 <springProperty name="APP_NAME" scope="context"
3               source="spring.application.name" defaultValue="spring-boot"/>
4 <!-- LogStash访问host -->
5 <springProperty name="LOG_STASH_HOST" scope="context"
6               source="logstash.host" defaultValue="localhost"/>
```

## 2.3 filter

在Logback中有两种不同的过滤器，用来过滤日志输出。

**ThresholdFilter**：临界值过滤器，过滤掉低于指定临界值的日志，比如下面的配置将过滤掉所有低于INFO级别的日志。

```
1 <filter class="ch.qos.logback.classic.filter.ThresholdFilter">
2   <level>INFO</level>
3 </filter>
```

**LevelFilter**：级别过滤器，根据日志级别进行过滤，比如下面的配置将过滤掉所有非ERROR级别的日志。

```
1 <filter class="ch.qos.logback.classic.filter.LevelFilter">
2   <level>ERROR</level>
3   <onMatch>ACCEPT</onMatch>
4   <onMismatch>DENY</onMismatch>
5 </filter>
```

## 2.4 appender

Appender可以用来控制日志的输出形式，主要有下面三种。

**ConsoleAppender**：控制日志输出到控制台的形式，比如在console-appender.xml中定义的默认控制台输出。

```

1 <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
2     <encoder>
3         <pattern>${CONSOLE_LOG_PATTERN}</pattern>
4     </encoder>
5 </appender>

```

**RollingFileAppender:** 控制日志输出到文件的形式，可以控制日志文件生成策略，比如文件名称格式、超过多大重新生成文件以及删除超过多少天的文件。

```

1 <!-- ERROR日志输出到文件 -->
2 <appender name="FILE_ERROR"
3     class="ch.qos.logback.core.rolling.RollingFileAppender">
4     <rollingPolicy
5         class="ch.qos.logback.core.rolling.SizeAndTimeBasedRollingPolicy">
6         <!-- 设置文件命名格式 -->
7         <fileNamePattern>${LOG_FILE_PATH}/error/${APP_NAME}-%d{yyyy-MM-dd}-
8         %i.log</fileNamePattern>
9         <!-- 设置日志文件大小，超过就重新生成文件，默认10M -->
10        <maxFileSize>${LOG_FILE_MAX_SIZE:-10MB}</maxFileSize>
11        <!-- 日志文件保留天数，默认30天 -->
12        <maxHistory>${LOG_FILE_MAX_HISTORY:-30}</maxHistory>
13    </rollingPolicy>
14 </appender>

```

**LogstashTcpSocketAppender:** 控制日志输出到Logstash的形式，可以用来配置Logstash的地址、访问策略以及日志的格式。

```

1 <!-- DEBUG日志输出到LogStash -->
2 <appender name="LOG_STASH_DEBUG"
3     class="net.logstash.logback.appender.LogstashTcpSocketAppender">
4     <filter class="ch.qos.logback.classic.filter.ThresholdFilter">
5         <level>DEBUG</level>
6     </filter>
7     <destination>${LOG_STASH_HOST}:4560</destination>
8     <encoder charset="UTF-8"
9         class="net.logstash.logback.encoder.LoggingEventCompositeJsonEncoder">
10        <providers>
11            <timestamp>
12                <timeZone>Asia/Shanghai</timeZone>
13            </timestamp>
14            <!-- 自定义日志输出格式 -->
15            <pattern>
16                <pattern>
17                    {
18                        "project": "${PROJ_NAME:-}",
19                        "level": "%level",
20                        "service": "${APP_NAME:-}",
21                        "pid": "${PID:-}",
22                        "thread": "%thread",
23                        "class": "%logger",
24                        "message": "%message",
25                        "stack_trace": "%exception{20}"
26                    }
27                </pattern>
28            </pattern>
29        </providers>
30    </encoder>
31 </appender>

```

```

26         </pattern>
27     </providers>
28 </encoder>
29 <!-- 当有多个LogStash服务时，设置访问策略为轮询 -->
30 <connectionStrategy>
31     <roundRobin>
32         <connectionTTL>5 minutes</connectionTTL>
33     </roundRobin>
34 </connectionStrategy>
35 </appender>

```

## 2.5 logger

只有配置到logger节点上的appender才会被使用，logger用于配置哪种条件下的日志被打印，root是一种特殊的appender，下面介绍下日志划分的条件。

- 调试日志：所有的DEBUG级别以上日志；
- 错误日志：所有的ERROR级别日志；
- 业务日志：com.zeroone.star.projectlog包下的所有DEBUG级别以上日志；
- 记录日志：com.zeroone.star.projectlog.component.WebLogAspect类下所有DEBUG级别以上日志，该类是统计接口访问信息的AOP切面类。

还有一些使用框架内部的日志，DEBUG级别的日志对我们并没有啥用处，都可以设置为了INFO以上级别。

```

1 <!-- 控制框架输出日志 -->
2 <logger name="org.slf4j" level="INFO" />
3 <logger name="springfox" level="INFO" />
4 <logger name="io.swagger" level="INFO" />
5 <logger name="org.springframework" level="INFO" />
6 <logger name="org.hibernate.validator" level="INFO" />

```

## 3 Logstash配置

接下来我们需要配置下Logstash，让它可以分场景收集不同的日志，下面详细介绍下使用到的配置。

```

1 input {
2     tcp {
3         mode => "server"
4         host => "0.0.0.0"
5         port => 4560
6         codec => json_lines
7         type => "debug"
8     }
9     tcp {
10        mode => "server"
11        host => "0.0.0.0"
12        port => 4561
13        codec => json_lines
14        type => "error"
15    }
16    tcp {
17        mode => "server"

```



```

18     host => "0.0.0.0"
19     port => 4562
20     codec => json_lines
21     type => "business"
22 }
23 tcp {
24     mode => "server"
25     host => "0.0.0.0"
26     port => 4563
27     codec => json_lines
28     type => "record"
29 }
30 }
31 filter{
32     if [type] == "record" {
33         mutate {
34             remove_field => "port"
35             remove_field => "host"
36             remove_field => "@version"
37         }
38         json {
39             source => "message"
40             remove_field => ["message"]
41         }
42     }
43 }
44 output {
45     elasticsearch {
46         hosts => "http://elasticsearch:9200"
47         action => "index"
48         codec => json
49         index => "project-%{type}-%{+YYYY.MM.dd}"
50         template_name => "project"
51     }
52 }

```

### 3.1 配置要点

- input: 使用不同端口收集不同类型的日志，从4560~4563开启四个端口；
- filter: 对于记录类型的日志，直接将JSON格式的message转化到source中去，便于搜索查看；
- output: 按类型、时间自定义索引格式。

## 4 Spring Boot配置

在Spring Boot中的配置可以直接用来覆盖Logback中的配置，比如`logging.level.root`就可以覆盖`<root>`节点中的`level`配置。

开发环境配置：`application-dev.yml`

```

1 logstash:
2   host: 192.168.220.127
3 logging:
4   level:
5     root: debug

```

测试环境配置: application-test.yml

```
1 logstash:
2   host: 192.168.220.128
3 logging:
4   level:
5     root: debug
```

生产环境配置: application-prod.yml

```
1 logstash:
2   host: 192.168.220.129
3 logging:
4   level:
5     root: info
```

## 5 Spring Boot集成案例

### 5.1 添加依赖

主要依赖

```
1 <!-- spring web mvc -->
2 <dependency>
3   <groupId>org.springframework.boot</groupId>
4   <artifactId>spring-boot-starter-web</artifactId>
5 </dependency>
6 <!-- spring boot aop -->
7 <dependency>
8   <groupId>org.springframework.boot</groupId>
9   <artifactId>spring-boot-starter-aop</artifactId>
10 </dependency>
11 <!-- logstash logback encoder -->
12 <dependency>
13   <groupId>net.logstash.logback</groupId>
14   <artifactId>logstash-logback-encoder</artifactId>
15 </dependency>
16 <!-- lombok -->
17 <dependency>
18   <groupId>org.projectlombok</groupId>
19   <artifactId>lombok</artifactId>
20   <optional>true</optional>
21 </dependency>
22 <!-- hutool -->
23 <dependency>
24   <groupId>cn.hutool</groupId>
25   <artifactId>hutool-all</artifactId>
26 </dependency>
27 <!-- knife4j -->
28 <dependency>
29   <groupId>com.github.xiaoymin</groupId>
30   <artifactId>knife4j-openapi3-jakarta-spring-boot-starter</artifactId>
31 </dependency>
```

## 5.2 添加logback配置

参考：[1 分场景配置](#)

## 5.3 修改项目配置

下面是application.yml配置

```
1 application:
2   title: 测试项目
3 spring:
4   application:
5     name: PROJECT-LOG
6   profiles:
7     active: dev
8 server:
9   port: 8080
```

多环境配置，参考[4 Spring Boot配置](#)

## 5.4 关键代码

### 5.4.1 访问记录对象

创建一个访问记录类，用于记录访问。

```
1 @Data
2 public class WebLog {
3     /**
4      * 操作描述
5      */
6     private String description;
7     /**
8      * 操作用户
9      */
10    private String username;
11    /**
12     * 操作时间
13     */
14    private Long startTime;
15    /**
16     * 消耗时间
17     */
18    private Integer spendTime;
19    /**
20     * 根路径
21     */
22    private String basePath;
23    /**
24     * URI
25     */
26    private String uri;
27    /**
```

```

28     * URL
29     */
30     private String url;
31     /**
32     * 请求类型
33     */
34     private String method;
35     /**
36     * IP地址
37     */
38     private String ip;
39     /**
40     * 请求参数
41     */
42     private Object parameter;
43     /**
44     * 请求返回的结果
45     */
46     private Object result;
47 }

```

### 5.4.2 AOP切面

书写一个AOP切面拦截请求，生成请求记录日志

```

1  @Aspect
2  @Component
3  @Order(1)
4  public class WebLogAspect {
5      private static final Logger LOGGER =
6      LoggerFactory.getLogger(WebLogAspect.class);
7      @Pointcut("execution(public * com.zeroone.star.projectlog.controller.*.*
8      (...))")
9      public void webLog() {
10     }
11     @Around("webLog()")
12     public Object doAround(ProceedingJoinPoint joinPoint) throws Throwable {
13         long startTime = System.currentTimeMillis();
14         //获取当前请求对象
15         ServletRequestAttributes attributes = (ServletRequestAttributes)
16         RequestContextHolder.getRequestAttributes();
17         HttpServletRequest request =
18         Objects.requireNonNull(attributes).getRequest();
19         //记录请求信息
20         WebLog webLog = new WebLog();
21         Object result = joinPoint.proceed();
22         Signature signature = joinPoint.getSignature();
23         MethodSignature methodSignature = (MethodSignature) signature;
24         Method method = methodSignature.getMethod();
25         if (method.isAnnotationPresent(Operation.class)) {
26             Operation apiOperation = method.getAnnotation(Operation.class);
27             webLog.setDescription(apiOperation.summary());
28         }
29         long endTime = System.currentTimeMillis();
30         String urlStr = request.getRequestURL().toString();

```

```

27     webLog.setBasePath(StrUtil.removeSuffix(urlStr,
URLUtil.url(urlStr).getPath()));
28     webLog.setIp(request.getRemoteUser());
29     webLog.setMethod(request.getMethod());
30     webLog.setParameter(getParameter(method, joinPoint.getArgs()));
31     webLog.setResult(result);
32     webLog.setSpendTime((int) (endTime - startTime));
33     webLog.setStartTime(startTime);
34     webLog.setUri(request.getRequestURI());
35     webLog.setUrl(request.getRequestURL().toString());
36     Map<String, Object> logMap = new HashMap<>(5);
37     logMap.put("url", webLog.getUrl());
38     logMap.put("method", webLog.getMethod());
39     logMap.put("parameter", webLog.getParameter());
40     logMap.put("spendTime", webLog.getSpendTime());
41     logMap.put("description", webLog.getDescription());
42     LOGGER.info(Markers.appendEntries(logMap),
JSONUtil.parse(webLog).toString());
43     return result;
44 }
45 /**
46  * 根据方法和传入的参数获取请求参数
47  */
48 private Object getParameter(Method method, Object[] args) {
49     List<Object> argList = new ArrayList<>();
50     Parameter[] parameters = method.getParameters();
51     for (int i = 0; i < parameters.length; i++) {
52         //将RequestBody注解修饰的参数作为请求参数
53         RequestBody requestBody =
parameters[i].getAnnotation(RequestBody.class);
54         if (requestBody != null) {
55             argList.add(args[i]);
56         }
57         //将RequestParam注解修饰的参数作为请求参数
58         RequestParam requestParam =
parameters[i].getAnnotation(RequestParam.class);
59         String clsName = args[i].getClass().getName();
60         if (requestParam != null) {
61             Map<String, Object> map = new HashMap<>(1);
62             String key = parameters[i].getName();
63             if (StringUtils.hasText(requestParam.value())) {
64                 key = requestParam.value();
65             }
66             map.put(key, args[i]);
67             argList.add(map);
68         }
69         //处理对象类型的RequestParam参数
70         else if (clsName.contains(".dto.") || clsName.contains(".query.")) {
71             argList.add(args[i]);
72         }
73     }
74     if (argList.size() == 0) {
75         return null;
76     } else if (argList.size() == 1) {
77         return argList.get(0);
78     } else {
79         return argList;
80     }

```

```
81     }
82 }
```

### 5.4.3 Swagger配置

参考项目演示示例代码中，我们集成的是knife4j

### 5.4.4 其他领域模型

其他领域模型类参考示例源码

### 5.4.5 测试控制器

书写一个测试控制器

```
1  @RestController
2  @RequestMapping("/test")
3  @Tag(name = "elk", description = "elk测试接口")
4  public class TestController {
5
6      @Operation(summary = "获取所有")
7      @GetMapping("query-all")
8      public JsonVO<PageVO<SampleVO>> queryAll() {
9          List<SampleVO> voList = new ArrayList<>(10);
10         for (int i = 0; i < 10; i++) {
11             SampleVO sampleVO = new SampleVO();
12             sampleVO.setId(i + 1);
13             sampleVO.setAge(11);
14             sampleVO.setName("李四-" + (i + 1));
15             sampleVO.setSex("男");
16             voList.add(sampleVO);
17         }
18         PageVO<SampleVO> pv = new PageVO<>(1L, 12L, 10L, 1L, voList);
19         return JsonVO.success(pv);
20     }
21
22     @Operation(summary = "添加数据")
23     @PostMapping("add")
24     public JsonVO<Long> add(SampleDTO data) {
25         return JsonVO.success(0L);
26     }
27
28     @Operation(summary = "修改数据")
29     @PutMapping("modify")
30     public JsonVO<Long> modify(SampleDTO data) {
31         return JsonVO.success(1L);
32     }
33
34     @Operation(summary = "删除数据")
35     @DeleteMapping("delete/{id}")
36     public JsonVO<Long> delete(@PathVariable("id") Long id) {
37         return JsonVO.success(id);
38     }
39
40     @Operation(summary = "条件查询")
41     @GetMapping("query")
```

```

42     public JsonVO<PageVO<SampleVO>> query(SampleQuery query) {
43         SampleVO sampleVO = new SampleVO();
44         sampleVO.setId(1);
45         sampleVO.setAge(11);
46         sampleVO.setName(query.getName());
47         if ("张三".equals(query.getName())) {
48             sampleVO.setSex("男");
49         } else {
50             sampleVO.setSex("女");
51         }
52         List<SampleVO> voList = new ArrayList<>(1);
53         voList.add(sampleVO);
54         PageVO<SampleVO> pv = new PageVO<>(1L, 12L, 1L, 1L, voList);
55         return JsonVO.success(pv);
56     }
57 }

```

## 5.5 启动项目

启动项目，观察控制台是否有报错，如果没有报错，访问API文档页面，<http://localhost:8080/doc.html>

看到如下图所示的页面

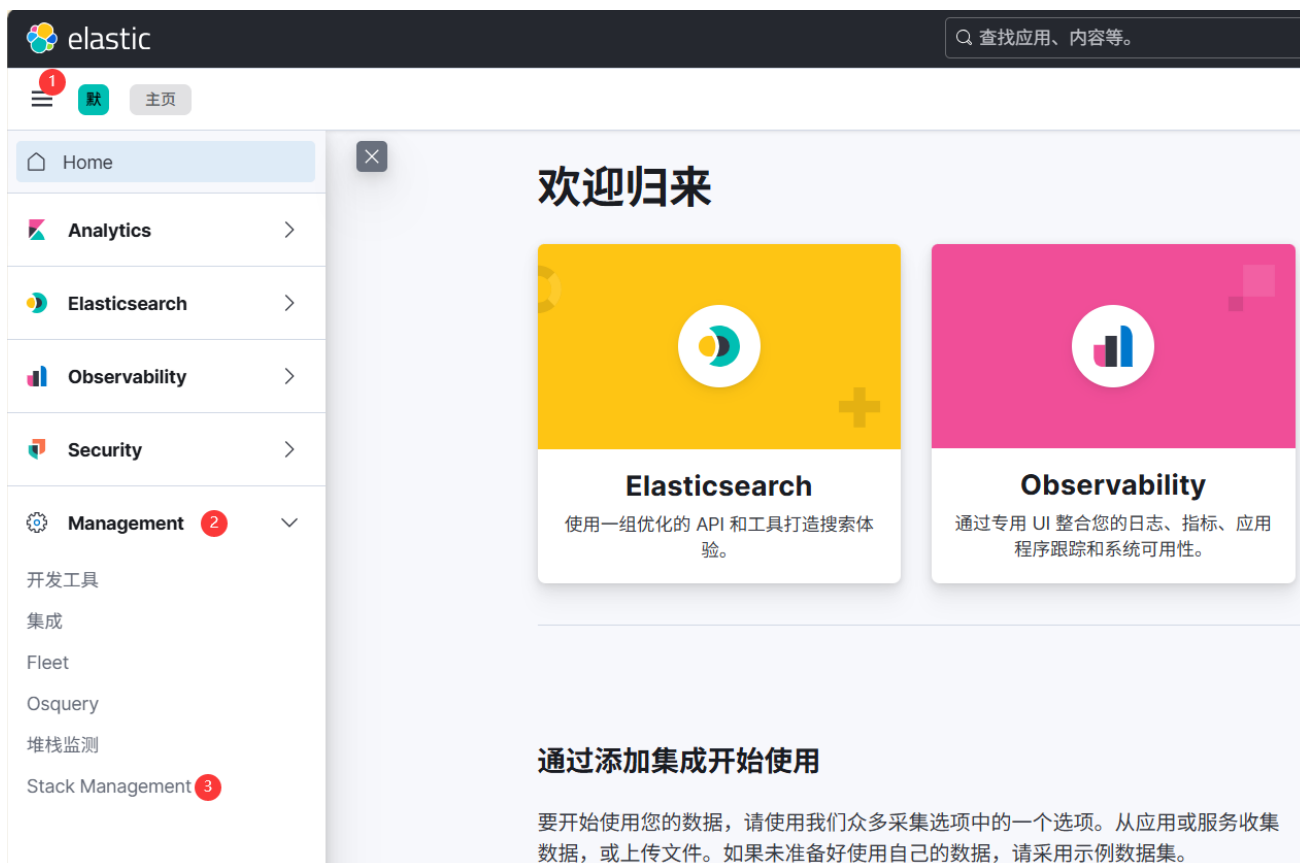


|            |                                                                                                                                                          |      |   |        |   |     |   |     |   |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|------|---|--------|---|-----|---|-----|---|
| host       | localhost:8080                                                                                                                                           |      |   |        |   |     |   |     |   |
| basePath   | /                                                                                                                                                        |      |   |        |   |     |   |     |   |
| 服务Url      |                                                                                                                                                          |      |   |        |   |     |   |     |   |
| 分组名称       | default                                                                                                                                                  |      |   |        |   |     |   |     |   |
| 分组Url      | /v2/api-docs                                                                                                                                             |      |   |        |   |     |   |     |   |
| 分组location | /v2/api-docs                                                                                                                                             |      |   |        |   |     |   |     |   |
| 接口统计信息     | <table> <tr> <td>POST</td><td>1</td></tr> <tr> <td>DELETE</td><td>1</td></tr> <tr> <td>PUT</td><td>1</td></tr> <tr> <td>GET</td><td>2</td></tr> </table> | POST | 1 | DELETE | 1 | PUT | 1 | GET | 2 |
| POST       | 1                                                                                                                                                        |      |   |        |   |     |   |     |   |
| DELETE     | 1                                                                                                                                                        |      |   |        |   |     |   |     |   |
| PUT        | 1                                                                                                                                                        |      |   |        |   |     |   |     |   |
| GET        | 2                                                                                                                                                        |      |   |        |   |     |   |     |   |

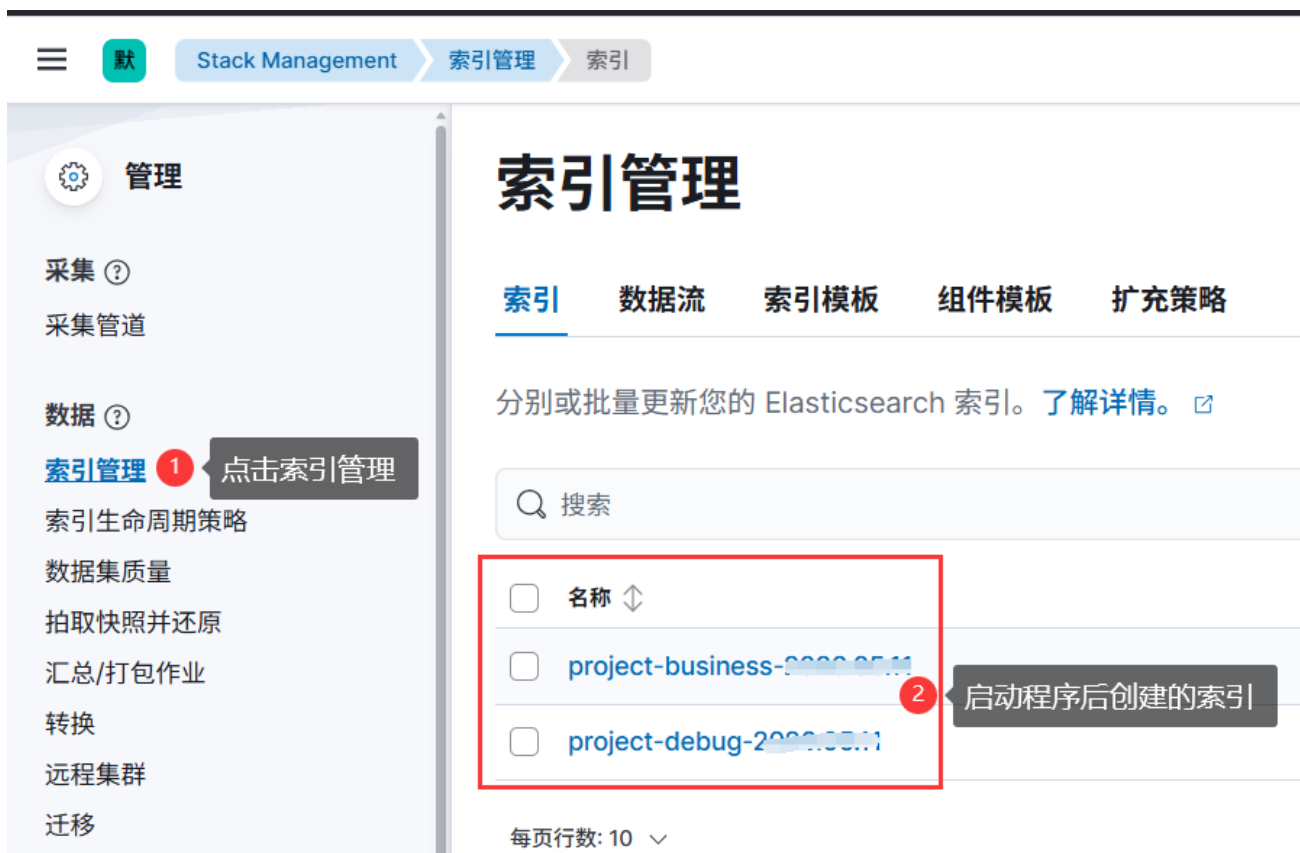
## 5.6 在Kibana中查看

访问你的Kibana，没有修改端口的访问地址为，<http://ip:5601>

### 5.6.1 查看索引



点击，可以查看已经产生的索引





## 5.6.2 数据视图

要使用Kibana查看索引详细信息，首先创建数据视图。



填写视图信息



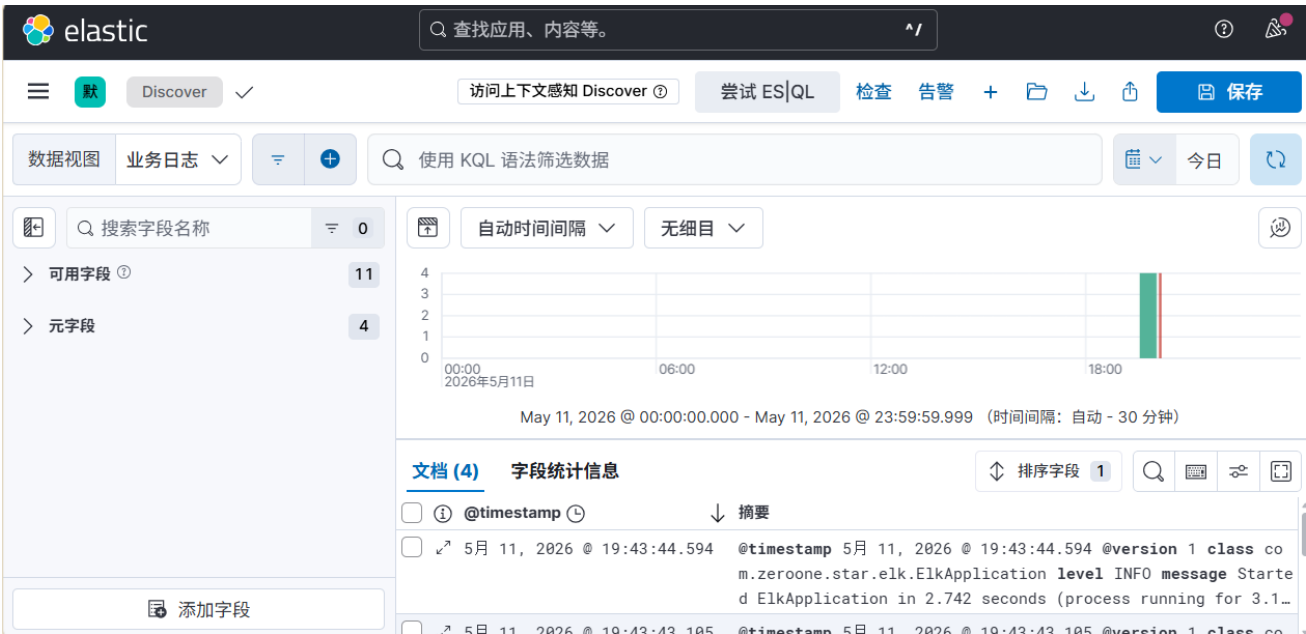
要创建其他数据视图可以如法炮制

### 5.6.3 查看日志

进入发现面板



进入面板后可以看到已经有一些日志了，如下图所示



然后再API页面访问接口，然后创建数据视图，就绪后可以在发现面板看到如下图所示的记录



测试异常，在任意接口中，第一句写入 `int er = 1 / 0 ;`，模拟产生异常

重启接口服务，然后访问修改的接口，然后创建数据视图，就绪后可以在发现面板中看到如下所示的记录



异常详情

使用 KQL 语法筛选数据

0

11

4

自动时间间隔

无细目

00:00  
2026年5月11日

06:00

May 11, 2026 @ 00:00:00.000

文档 (5)

字段统计信息

☐ ⓘ @timestamp ⌚

☐ ↗ 5月 11, 2026 @ 20:10:53.820

Servlet.service() for servlet [dispatcherServlet] in context with path [] threw exception [Request processing failed: java.lang.ArithmeticException: / by zero] with root cause

@timestamp 5月 11, 2026 @ 20:10:53.820 @version 1 class org.apache.catalina.core.ContainerBase.[Tomcat].[localhost].[/].[dispatcherServlet] level ERROR message Servlet.servic...